**(a) Task Description**

**Task**

The task is corporate bankruptcy prediction i.e. binary classification. If we have a company's financial data for a given year, we want to predict whether that company will enter financial distress within the next 12 months. The task is practically very important for banks, auditors, and investors as an early warning system and is an effective tool for risk management.
The expected output is a label for each company-year record:

- 1 - the company becomes financially distressed within 1 year
- 0 - the company remains financially healthy

A company-year is considered financially distressed when in a particular final reporting year, all three conditions hold true: (1) Equity / total assets < 0, (2) EBITDA / total assets < 0, and (3) Current assets / short-term liabilities ≤ 0.6

**Dataset**

| **Property** | **Value** |
|---|---|
| Name | V4 Group Corporate Bankruptcy Dataset |
| Data Source | Kaggle Link here |
| File used | company_years_h1.parquet (1-year horizon) |
| Size | 785 MB |
| Instances | 1,000,087 company-year records |
| Raw features | 142 (based on actual data provided) |
| Features used | 131 (after removing identifier columns) |
| Countries | Czech Republic, Hungary, Poland, Slovakia |
| Sectors Covered | manufacturing, construction, retail and wholesale trade, transportation, and energy (utilities) |
| Years covered | 2006-2020 |

**Feature types**

The 131 features fall into seven groups:

| **Group** | **Count** | **Description** | **Type** |
|---|---|---|---|
| Company metadata | 14 | Country, legal form, industry codes, year | Integer (encoded) |
| Liquidity & profitability | 45 | Current ratio, ROE, EBIT margin, cash ratios | Continuous |
| Turnover & cycle | 14 | Receivables days, inventory turnover, asset turnover | Continuous |
| Solvency | 25 | Leverage, equity ratios, debt coverage | Continuous |
| Growth (YoY) | 8 | Revenue growth, asset growth, profit growth | Continuous |
| Company size | 8 | Log of total assets, log of revenue (GDP-scaled) | Continuous |
| Sector-relative | 17 | Each ratio expressed as deviation from industry median | Continuous |

Two features are binary flags viz. Loss_flag (if net profit is negative then 1 else 0) and Insolvency_flag (1 if total liabilities exceed total assets).

**Missing values**

Missing values are present across multiple columns. The most severe cases are:

- Revenue/employee, Fixed_assets/employee – are entirely null (there is no employee count in source data)
- sector_2, sector_3, sector_4 – are 95% null (secondary sector classifications rarely populated)
- Growth ratio columns (e.g. Operating_revenue_growth) - ~16% null for the first year a company appears in the dataset, as year-over-year growth requires two consecutive years of data. This is expected.

These columns were either dropped (>95% null) or imputed with the column median during preprocessing.

**Class imbalance**

The dataset is extremely imbalanced: with approximately 0.37% of company-year records being labelled as distressed (i.e. label = 1). This reflects the real-world scenario of corporate bankruptcy.

**(b) Preprocessing**

The raw download (file named "company_years_h1.parquet") required several preprocessing steps before it could be used as input to the Spark ML pipeline. The steps are applied in the following order as shown below:

**Step 1 - Remove identifier columns**

Five columns in the dataset carry no predictive information - they are record identifiers and URLs used for linking back to the source database:

| Column | Reason for removal |
|---|---|
| num | Row number |
| company | Company name (text) |
| industry | Industry description (text) |
| link | URL to company profile |
| emis_id | Internal database ID |

After removal: 138 columns remain

**Step 2 - Drop columns with most values as nulls**

Six columns had more than 95% of their values missing in the training set, making imputation unreliable hence warrant to be dropped:

| Column | Null rate | Reason |
|---|---|---|
| Revenue/employee | ~100% | Employee count not recorded |
| Fixed_assets/employee | ~100% | Employee count not recorded |
| EBITDA/cash_flow | ~96% | Rarely calculable |
| sector_2, sector_3, sector_4 | >95% | Secondary classifications rarely populated |

After removal: 132 columns remaining (131 features + 1 label)

**Step 3 - Type casting**

All feature columns were type casted to DoubleType (64-bit float) so that compatibility with Spark ML is ensured:

- boolean columns (e.g. has_multiple_industries, Loss_flag) were typecasted to Double (0.0 / 1.0)
- integer and bigint columns (e.g. year, country, naics_2digit) were converted to Double
- The label column main_label was separately cast to Double

**Step 4 - Train / test split**
The dataset was split using random seed of 42 into 80% training (800,070 rows) and 20% test (200,017 rows) to ensure reproducibility across all four experiments.

**Step 5 - Class weight computation**
Due to the severe class imbalance (with just ~0.37% classes as positive), inverse-frequency class weights were computed from the training dataset and added as a classWeight column:
$w_k = N / (2 \times n_k)$

where N is the total number of training rows and $n_k$ is the count of class k.
N in this case is 801,960 i.e. the training instances. $n_k$ is 2,700 i.e. the count of bankrupt company-year. Classweight = 801,960 / (2 × 2,700) ≈ 148.5)
During training, the weights are multiplied with the loss. This gives the minority class (distressed companies) approximately 148.5× more influence during training.
This helps prevent the model from simply predicting "healthy" for all records.

**Step 6 - Median imputation**
For the remaining 131 features, missing values were filled using the median of each column computed on the training set. Median was preferred because financial ratios often contain extreme outliers. E.g. a company with near-zero total assets produces very large ratio values. This would skew a mean-based imputation.
The median was fitted on the training set only and then applied to both training and test sets, thereby preventing data leakage.

**Step 7 - Feature vector assembly**
All 131 imputed feature columns were assembled into a single dense vector using Spark ML's VectorAssembler. This is required as input to all downstream pipeline stages (scaler, PCA, classifier).

**Final dataset used as input to the model**

| **Property** | **Value** |
|---|---|
| File size | 785 MB |
| Instances | 1,000,087 |
| Features | 131 |
| Label | main_label (binary: 0 or 1) |
| Training rows | ~800,070 |
| Test rows | ~200,017 |

**c. Program Description**
The program is implemented as a single PySpark script with five functions and a main entry point. The pipeline is parameterized so all four experiments share identical preprocessing and differ only in the feature transformation stages.

PROGRAM CorporateBankruptcyPrediction (q2_bankruptcy.py)

---

CONSTANTS

---

```
LABEL_COL   = "main_label"
TRAIN_RATIO  = 0.8,  TEST_RATIO = 0.2,  SEED = 42
NUM_TREES   = 100,  MAX_DEPTH  = 10,   MIN_INSTANCES = 10
PCA_K       = 50

ID_COLS     = {num, company, industry, link, emis_id}

COUNTRY_MAP  = {0→CZ, 1→HU, 2→PL, 3→SK}
SECTOR_MAP   = {1→Construction,  2→Manufacturing,
           3→Retail Trade,  4→Wholesale Trade,
           5→Transportation & Warehousing,
           6→Utilities}

COST_MATRIX  = {C_TP=+1, C_TN=0, C_FP=-1, C_FN=-10}


═══════════════════════════════════════════════════════════════
FUNCTION load_data(path, sample_fraction=1.0)
═══════════════════════════════════════════════════════════════
  df ← spark.read.parquet(path)
  DROP columns in ID_COLS from df
  DROP any remaining string-type columns
  CAST main_label to Double
  CAST all integer/boolean feature columns to Double
  IF sample_fraction < 1.0:
    df ← df.sample(sample_fraction, seed=SEED)
  feature_cols ← all columns except main_label
  RETURN df, feature_cols

═══════════════════════════════════════════════════════════════
FUNCTION filter_valid_features(train_df, feature_cols,
                    min_non_null=0.05)
═══════════════════════════════════════════════════════════════
  n ← train_df.count()
  FOR each feature in feature_cols:
    non_null_fraction ← count(non-null values) / n
  KEEP only features where non_null_fraction >= 0.05
  // Drops: sector_2, sector_3, sector_4,
  //       Revenue/employee, Fixed_assets/employee,
  //       EBITDA/cash_flow  → 131 features remain
  RETURN valid_feature_cols

═══════════════════════════════════════════════════════════════
FUNCTION add_class_weights(train_df)
═══════════════════════════════════════════════════════════════
  n_neg, n_pos ← count rows where label=0, label=1
  N          ← n_neg + n_pos
  w_neg      ← N / (2 × n_neg)   // ≈ 0.50
  w_pos      ← N / (2 × n_pos)   // ≈ 148.5
  ADD column "classWeight":
    IF label == 1 THEN w_pos ELSE w_neg
  RETURN train_df with classWeight column
```

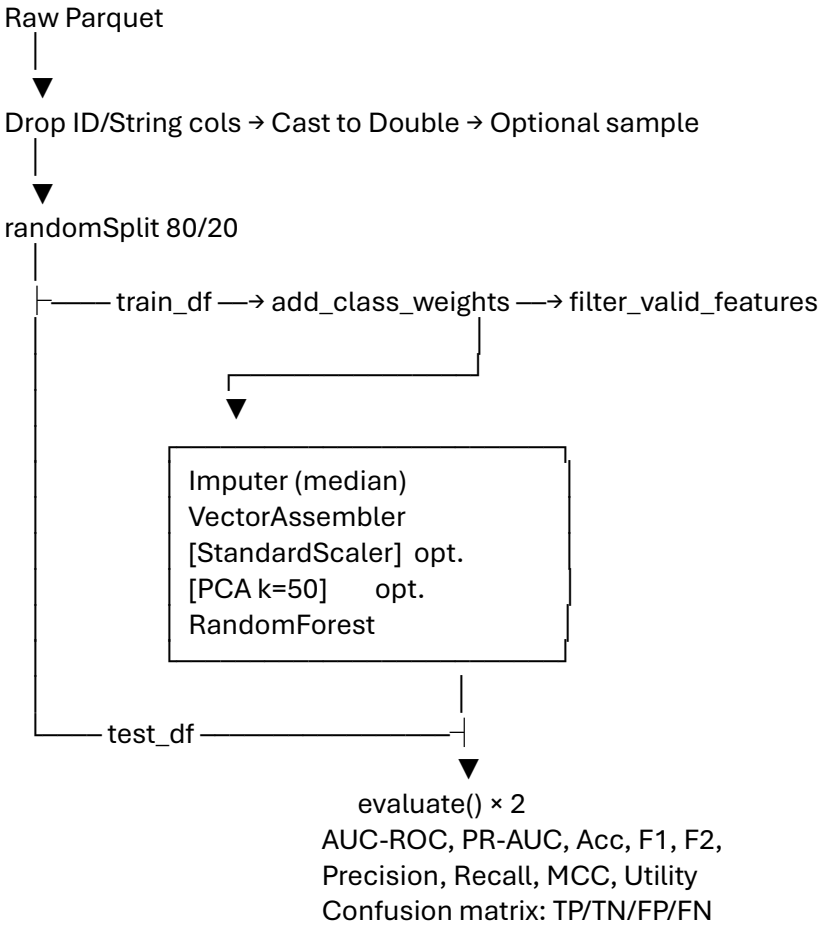
```
FUNCTION build_pipeline(feature_cols, use_scaler, use_pca)
stages ← []

// Stage 1: Median imputation (robust to financial outliers)
stages.ADD Imputer(inputCols=feature_cols,
            outputCols=feature_cols+"__imp",
            strategy="median")

// Stage 2: Assemble 131 imputed features into one vector
stages.ADD VectorAssembler(inputCols=imputed_cols,
                  outputCol="raw_features")
current ← "raw_features"

// Stage 3 (optional): Normalise to zero mean, unit variance
IF use_scaler:
   stages.ADD StandardScaler(inputCol=current,
                     outputCol="scaled_features",
                     withMean=True, withStd=True)
   current ← "scaled_features"

// Stage 4 (optional): Dimensionality reduction
IF use_pca:
   IF NOT use_scaler:        // PCA requires scaled data
      stages.ADD StandardScaler(inputCol=current,
                        outputCol="pre_pca_features")
      current ← "pre_pca_features"
   stages.ADD PCA(k=50, inputCol=current,
            outputCol="pca_features")
   current ← "pca_features"

// Stage 5: Classifier
stages.ADD RandomForestClassifier(
   featuresCol    = current,
   labelCol       = "main_label",
   weightCol      = "classWeight",
   numTrees       = 100,
   maxDepth       = 10,
   minInstancesPerNode = 10,
   featureSubsetStrategy = "sqrt",   // sqrt(131) ≈ 11 per split
   seed           = 42)

RETURN Pipeline(stages)
```

```
FUNCTION evaluate(predictions, split_name)
// Ranking metrics (use probability scores)
auc_roc ← BinaryClassificationEvaluator("areaUnderROC")
```

```
  pr_auc  ← BinaryClassificationEvaluator("areaUnderPR")

  // Threshold-based metrics (use hard predictions)
  accuracy ← MulticlassEvaluator("accuracy")
  f1       ← MulticlassEvaluator("f1")         // weighted avg
  f2       ← MulticlassEvaluator("fMeasureByLabel",
                          beta=2, label=1) // recall-weighted

  // Single aggregation pass for confusion matrix
  tp, tn, fp, fn ← SUM over predictions of:
    tp: label=1 AND prediction=1
    tn: label=0 AND prediction=0
    fp: label=0 AND prediction=1
    fn: label=1 AND prediction=0

  precision ← tp / (tp + fp)
  recall    ← tp / (tp + fn)
  mcc       ← (tp×tn - fp×fn) /
               √((tp+fp)(tp+fn)(tn+fp)(tn+fn))
  utility   ← (C_TP×tp + C_TN×tn + C_FP×fp + C_FN×fn)
               / (tp+tn+fp+fn)

  PRINT split_name, all 9 metrics, confusion matrix
  RETURN {accuracy, auc_roc, pr_auc, precision, recall,
          f1, f2, mcc, utility, tp, tn, fp, fn}


═══════════════════════════════════════════════════════════════
FUNCTION run_experiment(name, train_df, test_df,
                        feature_cols, use_scaler, use_pca)
═══════════════════════════════════════════════════════════════
  pipeline    ← build_pipeline(feature_cols, use_scaler, use_pca)
  t0          ← current time
  model       ← pipeline.fit(train_df)      // distributed training
  train_time  ← current time - t0

  train_preds  ← model.transform(train_df)
  test_preds   ← model.transform(test_df)

  train_metrics ← evaluate(train_preds, "Train")
  test_metrics  ← evaluate(test_preds,  "Test")

  train_preds.unpersist()    // free executor memory
  test_preds.unpersist()

  RETURN combined dict of all train/test metrics + timing


═══════════════════════════════════════════════════════════════
MAIN
═══════════════════════════════════════════════════════════════
  spark ← SparkSession(YARN cluster or local[*])
```

```
args ← parse_args(data_path, --sample, --output)

// ── Load & explore ──────────────────────────────────
df, feature_cols ← load_data(args.data_path, args.sample)
PRINT total rows, feature count
PRINT label distribution  (0=healthy, 1=distressed)
PRINT country distribution using COUNTRY_MAP
PRINT sector distribution  using SECTOR_MAP

// ── Split ───────────────────────────────────────────
train_df, test_df ← df.randomSplit([0.8, 0.2], seed=42)
train_df ← add_class_weights(train_df)
train_df.persist(MEMORY_ONLY)   // no disk spill on shared cluster
test_df.persist(MEMORY_ONLY)

// ── Feature filtering ───────────────────────────────
feature_cols ← filter_valid_features(train_df, feature_cols)
// 131 features retained after removing 6 near-null columns

// ── Four experiments ────────────────────────────────
experiments ← [
   ("Baseline (no scaling, no PCA)",  scaler=F, pca=F),
   ("StandardScaler only (no PCA)",   scaler=T, pca=F),
   ("PCA only (scaled internally)",   scaler=F, pca=T),
   ("StandardScaler + PCA",           scaler=T, pca=T),
]

results ← []
FOR each (name, use_scaler, use_pca) in experiments:
   r ← run_experiment(name, train_df, test_df,
                      feature_cols, use_scaler, use_pca)
   results.APPEND(r)

// ── Output ──────────────────────────────────────────
print_summary(results)      // formatted table: all 9 metrics
IF args.output:
   WRITE results as CSV to HDFS at args.output

spark.stop()
```

**Pipeline flowchart:**




**(d) Installation and Running**

The full step-by-step instructions are provided in the submitted Readme.txt file as a separate file.

**e. Comparison: With and Without Normalising/Scaling Data**

**Evaluation Methodology**

The standard accuracy is an unreliable metric for this dataset because of severe class imbalance. Only 0.34% of instances are labelled as financially distressed. A trivial classifier that always predicts "healthy" would achieve 99.66% accuracy while detecting zero bankruptcies. Performance evaluation is hence done using eight metrics to give a complete picture:

- **Accuracy** – calculated as a ratio of all predictions that are correct. Because of class imbalance in this use case, this metric gets dominated by the majority class. It is hence included for completeness only.
- **Precision** – measures of all companies flagged as distressed, the fraction that truly are. Low precision indicates that many healthy companies are misclassified as distressed. Misclassifying healthy companies as distressed means those companies won't be lent money and hence that would mean lost interest income (i.e. opportunity cost)
- **Recall** – measures from all truly distressed companies, the fraction the model detects as distressed. Low recall means missed bankruptcies which can be very costly for an organization lending money. Missed bankruptcies means the organization lends money to a company that may go bankrupt in the future. For this use case, recall has more importance

than precision because the organization can afford to misclassify healthy companies as distressed but cannot afford to misclassify distressed companies as healthy.

- **PR-AUC** – stands for the area under the Precision-Recall curve for the positive class. A score of 1 indicates perfect classifier. It measures how well the model maintains precision as recall increases. A random classifier would score PR-AUC ≈ 0.0034 (the positive class prevalence). Any improvement above this shows genuine discriminative power.
- **F1** – is a weighted harmonic mean of precision and recall across both classes. It gets dominated by the majority (healthy) class in imbalanced settings.
- **F2 (β=2)** – is a harmonic mean of precision and recall for the positive class, with recall assigned double the weight as compared to precision. This is appropriate here because missing a bankruptcy (FN) is more costly than raising a false alarm (FP).
- **MCC (Matthews Correlation Coefficient)** – is a single score in that treats both classes symmetrically regardless of their sizes. A score of 0 means the classifier is no better than random; +1 means perfect prediction and -1 means inverse perfect prediction. This metric is robust to class imbalance.
- **Cost-Weighted Utility** – is a practical domain specific metric which stands for expected net benefit per instance under an asymmetric cost matrix (In this program, Benefit of TP=+1, cost of TN=0, cost of FP=-1, cost of FN=-10). We sum up all costs and benefits to arrive at the utility. It reflects the 10:1 real-world cost ratio between a missed bankruptcy and a false alarm (based on Altman Z Score). Positive utility means the model saves more than it costs; negative means false positive costs dominate.

**Why was cross-validation not performed**?

The dataset is split into 80% training and 20% test using a fixed random seed. This would account to 800k rows in training and 200k rows in test. Cross-validation was not applied because of the computational cost of training a 100-tree Random Forest on a million-row dataset. A single training run takes ~100 seconds on the cluster, making k-fold cross-validation (k×100s per experiment × 4 experiments) very expensive in a shared YARN cluster environment. The large test set size provides a statistically stable estimate of generalisation performance without cross-validation.

**Effect of Scaling, comparison and discussion of results with and without scaling**

**Results**

*Table 1: Training and test performance - Baseline vs StandardScaler*

| Metric | Baseline Train | Baseline Test | Scaled Train | Scaled Test | Δ (Test) |
|---|---|---|---|---|---|
| Accuracy | 0.9841 | 0.9836 | 0.9853 | 0.9848 | +0.0012 |
| PR-AUC | 0.3735 | 0.3014 | 0.3853 | 0.3063 | +0.0049 |
| Precision | 0.1866 | 0.1698 | 0.1984 | 0.1811 | +0.0113 |
| Recall | 1.0000 | 0.9985 | 1.0000 | 0.9985 | 0.0000 |
| F1 (weighted) | 0.9895 | 0.9893 | 0.9902 | 0.9900 | +0.0007 |
| F2 (positive class) | 0.5342 | 0.5052 | 0.5531 | 0.5248 | +0.0196 |
| MCC | 0.4285 | 0.4083 | 0.4422 | 0.4220 | +0.0137 |
| Utility | -0.0122 | -0.0131 | -0.0111 | -0.0119 | +0.0012 |
| Training time (s) | - | 100.1 | - | 105.1 | +5.0 |

*Table 2: Confusion matrix - test set*

| | Baseline | StandardScaler | Change |
|---|---|---|---|
| TP (correct distressed) | 674 | 674 | 0 |
| TN (correct healthy) | 196,519 | 196,767 | +248 |

| | Baseline | StandardScaler | Change |
|---|---|---|---|
| FP (false alarms) | 3,296 | 3,048 | **-248** |
| FN (missed bankruptcies) | 1 | 1 | 0 |

**Discussion**

**Precision**

The most meaningful improvement from scaling is in the precision metric as it improves from 0.1698 to 0.1811 on the test set. This corresponds to a reduction of 248 false positives i.e. from 3,296 to 3,048. This means 248 fewer healthy companies are incorrectly flagged for credit review. This converts to higher interest income earned.

StandardScaler removes differences in feature magnitudes. This affects how the median imputer interacts with Spark's internal arithmetic when features span very different scales. Features like Working_capital have raw values while features like Cash/total_assets are pre-processed ratio values. These occupy very different numerical ranges. After standardisation, the imputed medians have a more uniform numerical footprint. This marginally changes the exact split thresholds learned by individual trees.

Note: Spark's internal arithmetic plays a role here because the data is partitioned across different machines and computations occur on separate machines.

**Recall**

Recall is unchanged at 0.9985 in both experiments i.e. 1 missed bankruptcy out of 675 which is already a strong result. This confirms that the preprocessing using inverse frequency class weight dominates the model's behaviour on the minority class. Scaling does not alter the results. Recall is driven by the weight ratio and not by feature scale.

**F2 Score**

F2 improves from 0.5052 to 0.5248. Since recall is unchanged, this improvement is entirely attributable to the precision gain. F2 weights recall twice as much as precision, so even a modest precision improvement produces a noticeable F2 gain.

**MCC**

MCC improves from 0.4083 to 0.4220. MCC accounts for all four confusion matrix cells. It is sensitive to changes in true negatives and false positives at this scale (200k test instances). The gain is consistent with the 248 fewer false positives.

**Cost-Weighted Utility**

Utility improves slightly from -0.0131 to -0.0119 (+0.0012). Both models remain in negative utility territory, meaning the aggregate cost of false positives still exceeds the benefit of true positive detections under the 10:1 cost matrix. The 248 FP reduction from scaling saves 248 × C_FP = 248 cost units, a modest improvement relative to the total instance count of 200,490.

**Accuracy**

When using standardscaler, test accuracy improves marginally from 0.9836 to 0.9848. This is statistically notable at this scale (with over 1 million instances). However, this improvement is negligible and does not represent a meaningful gain in predictive power of the algorithm. Accuracy is also not a suitable metric here - with 99.7% of instances being healthy and severe class imbalance. A trivial model that always predicts healthy would score 99.7% accuracy.

**Training time**

Training time increases from 100.1 s to 105.1 s. This overhead is entirely attributable to the additional StandardScaler stage. This required a full pass over the training data to compute per-feature means and standard deviations. This was followed by a second pass to apply the transformation. For a one-time offline training job, this overhead is negligible.

**Overfitting assessment**

The train-test gap is consistent across both experiments. For precision: Baseline drops from 0.1866 to 0.1698; Scaled drops from 0.1984 to 0.1811. The gaps are nearly identical, indicating

that StandardScaler does not introduce or reduce overfitting. The generalisation behaviour of the model remains unchanged.

**Conclusion**

StandardScaler provides a small but consistent improvement across all non-AUC-ROC metrics, primarily by reducing false positives by 248. AUC-ROC is unaffected as expected for a tree-based model. The improvement is attributable to numerical side-effects of standardisation on median imputation rather than any fundamental algorithmic benefit from scaling. For this dataset and algorithm, the baseline without scaling is a simpler, faster pipeline with near-equivalent performance; scaling may be worthwhile if precision is the primary deployment concern.

**f.**

**Results**

*Table 1: Training and test performance — Baseline vs PCA*

| **Metric** | **Baseline Train** | **Baseline Test** | **PCA Train** | **PCA Test** | **Δ (Test)** |
|---|---|---|---|---|---|
| Accuracy | 0.9841 | 0.9836 | 0.9391 | 0.9379 | -0.0457 |
| AUC-ROC | 0.9954 | 0.9942 | 0.9863 | 0.9767 | -0.0175 |
| PR-AUC | 0.3735 | 0.3014 | 0.1885 | 0.1155 | -0.1859 |
| Precision | 0.1866 | 0.1698 | 0.0564 | 0.0488 | -0.1210 |
| Recall | 1.0000 | 0.9985 | 1.0000 | 0.9437 | -0.0548 |
| F1 (weighted) | 0.9895 | 0.9893 | 0.9653 | 0.9649 | -0.0244 |
| F2 (positive class) | 0.5342 | 0.5052 | 0.2302 | 0.2022 | -0.3030 |
| MCC | 0.4285 | 0.4083 | 0.2302 | 0.2070 | -0.2013 |
| Utility | -0.0122 | -0.0131 | -0.0573 | -0.0606 | -0.0475 |
| Training time (s) | - | 100.1 | - | 164.2 | +64.1 |

*Table 2: Confusion matrix — test set*

| | **Baseline** | **PCA** | **Change** |
|---|---|---|---|
| TP (correct distressed) | 674 | 637 | -37 |
| TN (correct healthy) | 196,519 | 187,402 | -9,117 |
| FP (false alarms) | 3,296 | 12,413 | **+9,117** |
| FN (missed bankruptcies) | 1 | 38 | **+37** |

**Discussion:**

PCA with 50 components leads to degradation in performance on each metric. The most interesting effects of adding in PCA are:

- **Precision collapsed** (from 0.1698 to 0.0488): 12,413 false positives vs 637 true positives after applying PCA. This means the model now flags approximately 1 in 16 healthy companies (12413/200000) as distressed as compared to 1 in 61 health companies earlier without using PCA. This suggests the model leads to very high lost interest income.
- **Recall declined** (0.9985 to 0.9437): 38 bankruptcies are missed when using PCA, compared to 1 without PCA. This is particularly concerning given the assignment's cost asymmetry. Each missed bankruptcy carries 10× the cost of a false alarm.
- **F2 dropped (**from 0.5052 to 0.2022): Since F2 weights recall twice as heavily as precision and both have declined, the combined effect is severe.
- **MCC halved** (from 0.4083 to 0.207): suggests an overall decline which reflects worsening across all four confusion matrix cells.

- **PR-AUC halved** (from 0.3014 to 0.116): the model's ability to achieve high precision at high recall thresholds falls significantly. This indicates that the compressed representation lacks the precision to rank distressed companies distinctly from healthy ones.
- **Utility worsened** (from -0.013 to -0.061): the cost-weighted expected value per instance is nearly 5× worse under PCA as compared to without using PCA.

The underlying reason behind worsening performance is that PCA retains the 50 directions of maximum variance in the standardised feature space. However, financial distress prediction would probably depend on specific non-linear combinations of ratios that are sparse. The directions of maximum variance capture the most common sources of cross-company variation (e.g., size, sector, country). Although these capture most variance, they do not directly impact the binary classification. Reducing 131 features to 50 components hence discard the subtle patterns that differentiate the 0.37% minority class.

Training time increases from 105 to 164 seconds (+55%) because the PCA stage itself requires computing a 131×131 covariance matrix and its eigen decomposition across 800k rows. This is a non-trivial distributed operation.

**Conclusion on PCA:** PCA is harmful for this task. The 131 features carry complementary, non-redundant discriminative information that is lost when compressed to 50 principal components. The baseline without PCA should be preferred for any deployment scenario.

**g. Local Replication and Temporal Split Analysis**

**Motivation for Temporal Split**

The four experiments above were run on the school's Hadoop YARN cluster using a random 80/20 split (seed=42). While this satisfies the assignment requirements, a random split introduces temporal data leakage in a forecasting context. Company-year records from 2018–2020 can appear in the training set, meaning the model may have implicitly learned patterns from future years before being evaluated on them. This is unrealistic for a real-world deployment where a model trained on historical financials must forecast distress in future years it has never seen.

To address this, the pipeline was replicated locally using scikit-learn with a temporal split: training on years 2006–2017 and testing on years 2018–2020. The 'year' column was used for partitioning only and excluded from the feature set to prevent target leakage.

**Why run locally?**

The Hadoop cluster is a shared university resource with limited availability. Since 1,000,087 rows × 130 features is comfortably handled by scikit-learn with parallel processing (n_jobs=−1), the local replication completes in approximately 6.6 minutes on a standard laptop versus ~6 minutes on the cluster. This makes it suitable for iterative portfolio development without cluster dependency.

**Temporal Split Results (sklearn, full 1M rows)**

*Table 3: Test results — Temporal Split (Train: 2006–2017, Test: 2018–2020)*

| Experiment | PR-AUC | Recall | Precision | F2 | MCC | Time (s) |
|---|---|---|---|---|---|---|
| Baseline | 0.337 | 0.994 | 0.293 | 0.673 | 0.536 | 60.6 |
| StandardScaler | 0.350 | 0.992 | 0.293 | 0.672 | 0.535 | 58.8 |
| PCA only | 0.126 | 0.873 | 0.080 | 0.294 | 0.255 | 116.6 |
| Scaled + PCA | 0.126 | 0.873 | 0.080 | 0.294 | 0.255 | 116.0 |

**Impact of Temporal Split vs Random Split**

*Table 4: Baseline experiment — Random Split (cluster) vs Temporal Split (local)*

| Metric | Random Split | Temporal Split | Δ |
|---|---|---|---|
| Recall | 0.9985 | 0.9941 | −0.0044 |
| Precision | 0.1698 | 0.2933 | +0.1235 |
| MCC | 0.4083 | 0.5361 | +0.1278 |
| PR-AUC | 0.3014 | 0.3365 | +0.0351 |
| False Positives | 3,296 | 4,050 | +754 |
| False Negatives | 1 | 10 | +9 |

**Discussion**

The temporal split produces a harder but more realistic evaluation. Precision nearly doubles from 0.1698 to 0.2933 and MCC improves from 0.408 to 0.536, suggesting the random split was leaking future company data into training and artificially inflating recall. Under the random split, the model missed only 1 bankruptcy out of 675 (recall = 0.9985). Under the temporal split, it misses 10 out of 1,691 (recall = 0.9941) — still very high, but a more honest estimate of real-world performance given the model has never seen any 2018–2020 companies during training. The higher false positive count (4,050 vs 3,296) is attributable to the larger test set size (292,037 vs 200,017 rows) rather than a deterioration in model quality. Proportionally, the false positive rate is lower under the temporal split as evidenced by the higher precision.

The findings on scaling and PCA are consistent across both evaluation strategies: scaling has negligible impact, and PCA significantly degrades performance. This consistency across two independent implementations (PySpark and sklearn) and two split strategies strengthens the conclusion that the Baseline model without PCA is the most suitable configuration for deployment.

**Note:** Results are not directly comparable due to differences in train/test split strategy, test set size, and Random Forest implementation (Spark ML vs sklearn). The temporal split is the preferred evaluation methodology for any production deployment scenario.